# LLM-Serving Scheduler Ranking Portability Across Workloads, Operating Regions, and Metrics

SOROUSH VAHIDI, New Jersey Institute of Technology, USA

Scheduler rankings are conditional on the workload, operating region, metric and execution model used to obtain them. We present a reproducible comparison methodology and an empirical study of a frozen, trace-driven LLM-serving simulator corpus: three released trace sources, 120 windows, six operating regions and 13 executed policies, including an 11-policy primary panel. The 9,360 configurations each have two deterministic verification passes, not independent stochastic repetitions. For the primary arrival-normalized weighted-goodput metric, cross-source Kendall correlations range from 0.551 to 1.000. Of 990 pairwise-policy comparison records, 36 satisfy the saved practical-margin and interval criteria for a reversal; they span five policy pairs and all involve BurstGPT. A post-campaign cross-metric analysis yields a median correlation of 0.419 and deterministic top-choice agreement of 31.9%, with explicit tie and missing-observation qualifications. These characterize the historical simulator, not native serving rankings. Retrospective audits of the legacy GPU comparison and subsequent CPU conformance work leave faithful physical ranking transfer unverified. The results support reporting comparative rankings with their workload, metric, uncertainty and implementation boundaries.

CCS Concepts: • **Computing methodologies** → **Simulation evaluation**; • **Computer systems organization** → *Cloud computing*.

Additional Key Words and Phrases: LLM serving, scheduling, ranking portability, trace-driven simulation, performance evaluation

## 1 Introduction

A scheduler ranking is a conditional statement. It compares a fixed set of implementations under a workload, an operating region, an objective and an execution model. A ranking obtained from one trace cannot automatically be used to select a policy for another trace; nor does agreement on average throughput imply agreement on deadline attainment or tail latency. This paper studies how such comparative conclusions transfer, rather than introducing a new scheduler.

LLM serving makes this question concrete. Continuous batching and paged key–value (KV) memory management permit concurrent requests [9, 22]. Chunked prefill changes competition between prompt processing and decoding [2]; disaggregation separates their resource requirements [13, 24]. Production traces differ in arrival bursts, prompt lengths and output lengths [16, 19]. These established observations motivate testing the stability of a comparison across conditions. They do not establish which policies will reverse order, or even that reversals must occur in a particular evaluation.

We study the LLM-Serving Scheduler Portability Benchmark (LSSP), a frozen, trace-driven simulator campaign. Its primary corpus contains 120 windows from three separately released trace sources. A 13-policy execution panel, including 11 primary policies and two style approximations, is evaluated in six FIFO-referenced operating regions. There are 9,360 distinct configurations and two deterministic verification passes per configuration. The passes check repeatability of the simulator; they are not independent samples. Uncertainty is instead assessed across workload windows, subject to the dependence limitations in Section 10.

Author's Contact Information: Soroush Vahidi, sv96@njit.edu, New Jersey Institute of Technology, Newark, New Jersey, USA.

2026. ACM 2376-3647/2026/10-ART
https://doi.org/

The contribution is a reproducible evaluation methodology and an empirical characterization of *simulated* scheduler-ranking portability. The methodology combines a fixed comparison panel, matched policy outcomes, explicit metric eligibility, rank agreement, practical-margin reversal classification and a finite-corpus ranking-recovery analysis. Kendall correlation, bootstrap intervals and false-discovery correction are standard statistical tools, not a new algorithm or theoretical result. The scientific object is the reliability and scope of the comparative conclusion produced by their application.

The primary ANWG ranking correlations range from 0.551 to 1.000 across 18 source-pair/region comparisons. Of 990 pairwise-policy comparison records, 36 meet the saved practical-margin and confidence-interval reversal criteria; these span five policy pairs and all involve BurstGPT. Most comparisons therefore retain their order. A separately labeled post-campaign analysis shows a median cross-metric Kendall correlation of 0.419 and agreement of the deterministic top-choice rule in 316 of 990 conditions. Agreement and disagreement are both findings; neither is assumed in advance.

These are observations of historical simulator implementations. Later source and engineering audits identified limits in partial-prefill execution, native admission, KV allocation, preemption and execution acknowledgment. The historical GPU comparison also has metadata and replay limitations. It cannot establish faithful native-policy ranking transfer. Accordingly, this paper neither claims physical validation of its simulated reversals nor generalizes them to GPU architectures or parallel deployments. The distinction is central to the domain of validity, not a qualification added after interpreting the numerical results.

The original six research questions are retained in Section 3. Sections 4.2 and 5 define the model and experimental design; Sections 6, 7 and 8 separate operating-region behavior, cross-source portability and cross-metric portability. Robustness and practical implications precede an explicit account of threats to validity.

## 2 Background and Related Work

### 2.1 Serving mechanisms and workload variation

Prefill processes an input context; autoregressive decoding generates new tokens while retaining KV state. Iteration-level scheduling in Orca [22], PagedAttention in vLLM [9] and stall-free chunked prefill in Sarathi-Serve [2] address different batching and memory constraints. DistServe and Splitwise separate prefill and decode resources [13, 24]; Llumnix rebalances instances [17]; Mooncake organizes serving around KV state [14]. Fairness-aware token accounting [15], service-length-aware packing [8], learned ordering [7] and SLO-aware scheduling [4] illustrate why a single scalar ranking need not preserve every deployment objective. We compare historical simulator policies inspired by such mechanisms, not these systems as interchangeable production implementations.

BurstGPT, Azure and Bailian expose different production-derived request populations [3, 10, 19]. Characterization and generation work examines workload heterogeneity, including KV reuse and changes over time [11, 18, 20]. BurstGPT and Azure are separately released sources associated with Azure infrastructure; their comparison does not isolate a provider effect. Observed prompt lengths can also have different conversation/context semantics. LSSP retains these distinctions when interpreting portability.

### 2.2 Simulation and evaluation methodology

Vidur and LLMServingSim provide infrastructure for evaluating serving configurations [1, 5, 6]. LSSP does not claim a more accurate simulator. Its emphasis is the comparative conclusion induced by a fixed model, together with the restrictions on transporting that conclusion beyond the model.

Workload-dependent performance is established background. Work on serving workload selection [12], production-trace dispatching [21], and workload resampling in parallel job scheduling [23] provides close methodological precedents. The distinctive scope pursued here is a common, frozen panel with jointly reported source, region and metric comparisons, explicit reversal denominators, and uncertainty conditional on the available windows. It is not a priority claim that workload sensitivity or ranking comparison was previously unknown.

Kendall's tau-b measures ordinal concordance while accommodating ties; Spearman correlation and top-$k$ overlap provide complementary summaries. Neither a positive aggregate correlation nor a shared top choice rules out pair-specific reversals. Practical margins distinguish a numerically changed order from an operationally meaningful difference under a stated convention. These standard measures serve a reproducible comparison protocol. Our finite-corpus recovery study estimates stability relative to the frozen reference ranking, not a theorem about population sample complexity.

## 3 Research Questions and Portability Measures

We preserve the original six questions, including those not answered by adequate evidence in this study:

**RQ1.** How stable are scheduler rankings across independent workload sources?
**RQ2.** How stable are scheduler rankings under temporal, provider, and domain shifts?
**RQ3.** To what extent do rankings obtained on synthetic stress workloads transfer to rankings on independent real-trace-derived workloads?
**RQ4.** Which source-native observable workload characteristics are associated with cross-distribution scheduler rank reversals?
**RQ5.** How many independent workload windows are required before a comparative scheduler ranking becomes statistically stable?
**RQ6.** Do representative simulated scheduler rankings and cross-workload rank reversals reproduce on a real serving engine?

Here "independent sources" means separately released artifacts, not proven independence of providers or of every request. RQ1/RQ2 are the primary cross-source and temporal questions. RQ3 remains an engineering pilot, RQ4 a descriptive association analysis, RQ5 finite-reference recovery, and RQ6 unresolved as a faithful native-system comparison. Operating-region sensitivity is secondary. Cross-metric analysis is explicitly post-campaign; its prominence in this manuscript does not make it retrospectively preregistered.

### 3.1 Comparison object and statistical unit

Let $R(W, M, L)$ denote the ordering of a fixed policy panel under workload source or window set $W$, metric $M$, and operating region $L$. Policy outcomes are paired within the same window and region. For source-level rankings, a policy's defined outcomes are averaged over windows, after the two identical deterministic execution passes are collapsed. Comparisons between sources use different window populations; they are not paired request-by-request across sources. The window is the resampling unit. Non-overlap and deterministic sampling do not prove statistical independence of windows drawn from a shared production stream.

Tau-b is computed from the policy-level aggregate values, preserving equal values as ties. Top-$k$ summaries instead sort by value and then by policy identifier. In particular, the reported top-1 agreement is agreement of this deterministic tie-breaking rule, not overlap of all policies tied for the optimum. Exact-order recovery uses the same distinction. These conventions are retained for reproducibility and bound the interpretation of preferred-policy stability.

### 3.2 Metrics

Three metrics are always defined for every completed cell: completion fraction, weighted completion fraction, and Arrival-Normalized Weighted Goodput (ANWG, the primary metric). We formally define ANWG here because it is this benchmark's headline measure of system-level utility. For a set of arriving requests $R$, let $w_i \geq 1$ denote the priority weight assigned to request $i \in R$. Let $C \subseteq R$ be the subset of requests that successfully complete execution, and let $S \subseteq C$ be the subset of completed requests that also meet their latency deadline (SLO), with completion at or before the deadline. ANWG is defined as the priority-weighted sum of successfully attained deadlines divided by the total priority weight of all arrivals:

$$\text{ANWG} = \frac{\sum_{i \in S} w_i}{\sum_{i \in R} w_i} \tag{1}$$

Because the denominator is the sum over all *arrivals* rather than completions, rejected, dropped, unfinished, and SLO-missed requests all receive zero numerator credit while remaining in the denominator penalty. ANWG is always defined (the denominator is zero only if a workload window has no arrivals, which is prevented by construction); therefore, undefined-metric exclusions for ANWG are always exactly zero. A second group of metrics is defined only conditional on at least one request completing: SLO-violation rate, weighted goodput, mean/p95/p99 latency, and request/token throughput are each `NaN` if and only if completion fraction is exactly zero for that cell. Mean/p95 time-to-first-token (TTFT) is subject to a stricter, separately checked precondition (at least one completed request recorded a first-token time), because TTFT can be undefined even when completion fraction is positive. No undefined conditional-metric value is ever imputed as 0 or 1, in either direction, and never after inspecting which policy or source it would affect. Where a policy's value is undefined for a given (window, region, metric) condition, that policy is excluded from the ranking-derived statistic (Kendall's tau-b, Spearman's rho, top-$k$ overlap, pairwise reversal detection) for that specific condition and metric only – never scored as a loss or a tie, and never causing the whole condition to be dropped for policies that did complete. Every ranking-derived table reports its exclusion count (`n_conditions_excluded_for_undefined_metric`) alongside the statistic itself, since a source-concentrated exclusion rate is itself a reportable finding under RQ4, not a footnote.

### 3.3 Ranking-Uncertainty Quantification and Reversal Classification

Because the statistical unit of analysis is the workload window (§3.1), ranking-uncertainty is quantified by a whole-window block bootstrap: windows, not individual requests, are resampled with replacement (preserving each window's internal request sequence intact), 2,000 resamples are drawn per condition, and every reported confidence interval is a 95% percentile interval over those resamples, using a deterministic, recorded seed so every interval is exactly reproducible from the frozen outcome matrix alone. This block structure exists because the requests within a window are not independent observations of scheduler behavior – they share one arrival process, one prompt/output-length mixture, and one operating region – so resampling at the request level would understate uncertainty; the window is the design's resampling unit. Residual temporal dependence between windows may still affect interval coverage; it is not removed merely by treating requests within a window as a block.

Two further checks calibrate how the pairwise reversal statistics reported in §7.3 are interpreted, and it is important not to conflate them. The frozen output contains 66 Friedman omnibus records (11 metrics by six regions). Earlier manuscript text incorrectly described the omnibus test as unimplemented; this draft corrects that statement. The principal reversal findings below use the separately recorded pairwise procedure. Separately, because the panel and the (source, region,

metric) grid together generate many pairwise comparisons, every pairwise reversal test's two-sided bootstrap sign-test $p$-value – the intersection-union combination $\max(p_x, p_y)$ of the two compared conditions' sign-test $p$-values, read off the same resamples that back the ordinary CI rule below – is additionally checked against a Benjamini–Hochberg false-discovery-rate (FDR) correction at $q = 0.05$, applied per family, where a family is all pairwise policy comparisons within a single (metric, load-region) combination. This produces a separate, multiplicity-corrected `supported_after_fdr` flag on every record. In the frozen benchmark artifact, all 36 records that the five-class taxonomy below independently classifies as `SUPPORTED_PRACTICAL_REVERSAL` also satisfy `supported_after_fdr` – the FDR check removes no headline reversal in this dataset – but the two criteria are computed independently from each other and are not guaranteed to coincide in general.

A numerical sign change in a point estimate is not, by itself, a meaningful scheduler reversal from a systems standpoint: two policies can swap point-estimate order by a magnitude far smaller than the benchmark's own chosen practical margin, in which case treating it as a reversal would overstate what the benchmark actually supports. The 10% winning-margin threshold used throughout this test was fixed prospectively, before any reversal was observed, by reusing this project's own earlier discriminability-pilot criterion (`_relative_range_qualifies`, §5.8) rather than adopting a threshold chosen after seeing these results or asserted as a domain-standard cutoff; it is an operational convention for separating materially sized ordering changes from microscopic sign changes, not a claim that 10% is a universal practical-significance threshold for LLM serving, and every count labeled "practical" in this manuscript is conditional on it. Every pairwise (policy A, policy B) comparison at a given (source, region, metric) condition is therefore classified into exactly one of five preregistered classes, combining the practical-margin test with the *ordinary* (not FDR-adjusted) 95% percentile bootstrap-CI test defined above; the FDR check described above is a separate, additional multiplicity safeguard applied on top of this classification for headline reporting, not an input the classification itself depends on:

**`UNDEFINED_UNESTIMABLE`** At least one of the two policies' metric values is undefined for this condition (§3.2); no direction is estimated or claimed.

**`STABLE_NO_SIGN_CHANGE`** The two policies' relative order agrees across the compared conditions; no reversal occurred.

**`MICROSCOPIC_SIGN_CHANGE`** The point-estimate order reverses, but the winning margin fails to exceed 10% of the losing policy's value in *at least one* of the two compared conditions – this includes both the case where the margin is negligible on both sides and the asymmetric case where it exceeds 10% on one side but not the other. Either way the sign flip is reported here as numerically real but practically negligible or unreliable, never pooled with the classes below.

**`UNSUPPORTED_SIGN_CHANGE_WIDE_CI`** The point-estimate order reverses and the winning margin exceeds 10% of the losing policy's value in *both* compared conditions, but the *ordinary* (not FDR-adjusted) 95% bootstrap interval for the difference fails to exclude zero in *at least one* of the two conditions – the apparent reversal is not statistically distinguishable from bootstrap noise, at the preregistered confidence level, in at least one of the two conditions being compared.

**`SUPPORTED_PRACTICAL_REVERSAL`** The point-estimate order reverses *and* the winning margin exceeds 10% of the losing policy's value in both compared conditions *and* the *ordinary* (not FDR-adjusted) 95% bootstrap interval for the direction excludes zero in both conditions. Only this class is reported as a headline reversal in §7.3; the other four classes are always reported alongside it, never omitted as a residual category. Every record in this class additionally

satisfies the separate FDR-corrected `supported_after_fdr` check described above in the frozen benchmark artifact.

These five classes are mutually exclusive and exhaustive by construction: `classify_pairwise_reversal` in the frozen analysis code evaluates undefined-value, sign-change, margin, and confidence-interval conditions as a strict if/elif/else chain with an unconditional final branch, so every possible (policy A, policy B, condition) input reaches exactly one class, including the asymmetric-margin and single-sided-wide-CI cases described above; no already-computed reversal record is at risk of being silently dropped or misclassified by a gap in this taxonomy.

### 3.4 Aggregation, eligibility and reproducible interpretation

Write $y_{w,p,m}$ for the defined metric value of policy $p$ on window $w$ under metric $m$, with source and region fixed. Let $\mathcal{W}_{p,m}$ be the windows where that value is defined. The historical aggregate is

$$\bar{y}_{p,m} = \frac{1}{|\mathcal{W}_{p,m}|} \sum_{w \in \mathcal{W}_{p,m}} y_{w,p,m}. \tag{2}$$

If this set is empty, the policy has no aggregate for that condition. Lower-is-better values are negated for cross-metric ranking comparison. Missing values never become a zero, a worst rank, or a tied rank. A consequence is that two policy means can describe different observed window subsets. The pairwise bootstrap difference instead uses the intersection of windows with both policy outcomes. Agreement of complete conditions does not remove this conditioning difference.

For comparable policy-value vectors, let $C$ and $D$ be the numbers of concordant and discordant policy pairs, and $T_x, T_y$ the pairs tied only in the first or second vector. Kendall's tie-adjusted coefficient is

$$\tau_b = \frac{C - D}{\sqrt{(C + D + T_x)(C + D + T_y)}}. \tag{3}$$

A zero denominator leaves the coefficient undefined. A positive aggregate value allows individual discordant pairs. Sorting ties lexicographically for top-$k$ reporting is an implementation convention separate from this coefficient; it must not silently change the ties counted by tau-b.

For two conditions $x, y$ and policy pair $a, b$, the recorded differences are $d_x = \bar{y}_{a,x} - \bar{y}_{b,x}$ and $d_y = \bar{y}_{a,y} - \bar{y}_{b,y}$. The implementation requires both nonzero and opposite in sign before considering a reversal. Its relative margin divides the absolute difference by the absolute aggregate value of the losing policy in that condition. A zero losing value makes that margin unestimable; it is not an automatic practical-significance pass. Equal values are assigned zero difference and zero margin and do not constitute a sign change. These edge cases matter particularly for bounded utility metrics with many exact ties.

The saved procedure evaluates eligibility, sign, margin and then interval support in that order. Consequently, many stable or microscopic records have no bootstrap interval or $p$-value at all. The BH check is applied to tests that reach the statistical-support stage within the recorded family, not to invented $p$-values for every stable record. For the primary analysis the family is metric/region; for the cross-metric extension it is source/region/metric-pair. The reported flags are reproducible results of that procedure. Their interpretation is conditional on this screening, bootstrap construction and dependence structure; this paper does not prove unconditional false-discovery control for the entire research workflow.

The saved two-sided bootstrap tail fraction is computed from the same 2,000 resamples as the interval, and can equal zero at finite resolution. This is not evidence of zero probability of an incorrect scientific claim. Cross-source ranking bootstraps draw the two source window collections separately. Cross-metric bootstraps use a common sampled window sequence because both metrics describe

Table 1. Units in the historical study. Numerically equal totals need not refer to the same evidence.

| Quantity | Interpretation |
|---|---|
| 120 windows | 40 per source; 200 requests in each window |
| 9,360 configurations | 120 windows times six regions times 13 policies |
| 18,720 executions | Two deterministic passes per configuration |
| 18 primary ranking comparisons | Three source pairs times six regions |
| 990 primary reversal records | 18 conditions times 55 policy pairs |
| 990 cross-metric conditions | Three sources times six regions times 55 metric pairs |
| 54,450 metric disagreement records | 990 conditions times 55 policy pairs |
| 500 draws per recovery budget | Subsamples of a fixed reference corpus, not new executions |
| 240 legacy physical executions | Separate runner and fidelity limits; not included in the simulator matrix |

the same outcomes. Repetition IDs are never resampled as additional windows. M1 verifies saved intervals' classification logic and point estimates, but does not rerun these stochastic resampling calculations or substitute new interval estimates for the frozen ones.

### 3.5 Evidence denominators

The inferential unit remains the window. Comparison records reuse its outcomes many times; their counts describe coverage rather than sample independence. A change in the number of policy pairs or metrics changes these denominators without acquiring any additional workload evidence.

## 4 Evaluation Model and Assumptions

### 4.1 Scheduler Panel

The PRIMARY panel comprises 11 policies, executed alongside 2 additional `STYLE_APPROXIMATION` policies used only for a mandatory robustness check (13 policies executed in total): a classical arrival-order baseline (`fifo`, also the load-calibration reference policy), a deadline-aware policy (`edf`), a slack/urgency-aware policy (`least_laxity_first`), a service-length-aware policy (`estimated_service_time_first`), a fairness-aware policy (`weighted_fair_share`), a KV/memory-pressure-aware policy (`kv_constrained_online`), an admission-control/SLO-guard policy (`admission_control`), and four historically named simulator implementations of external mechanisms: vLLM-style token-budget continuous batching in both its pre-chunked-prefill and chunked-prefill forms (`vllm_faithful`, `vllm_chunked_prefill_faithful`), Sarathi-style chunked prefill (`sarathi_faithful`), and an SLO-aware scheduling policy in the SLAI/RAD family (`slai_faithful`). The two `STYLE_APPROXIMATION` robustness policies are a token-budget-proxy style approximation (`vllm_style_token_budget`) and an SLO-guard-proxy style approximation (`scorpio_style_slo_guard`); two further implementations requiring disaggregated/cross-instance simulator semantics (`distserve_faithful`, `llumnix_faithful`) are designated for a secondary stratum, never pooled with the single-GPU colocated PRIMARY leaderboard, and one candidate (`apt_serve_faithful`) is excluded entirely as `scaffolding_only` – not a validated reimplementation at the time of this freeze.

A policy enters the PRIMARY panel only if, at the time it was added to the comparability audit – frozen a full day before the discriminability pilot (§5.8) executed – it satisfies an outcome-independent selection rule: its mechanism predates this protocol's freeze, it occupies a mechanism family not already saturated by an existing panel member, it has adequate implementation fidelity and passes its own unit tests, it is simulator-semantically compatible with the single-GPU colocated execution model, and it is independently reproducible from a cited reference. No policy was added to or removed from the panel in response to the pilot's outcome, and mechanism families it identified as under-implemented (preemptive/multilevel scheduling in the FastServe sense, JITServe, LMetric) were deliberately left as documented future-work candidates rather than fast-tracked into the panel after seeing the pilot's BurstGPT collapse.

### 4.2 Execution Model

Each (window, policy, operating-region) cell is executed as a deterministic discrete-event simulation. The entities are requests, one or more simulated GPU resources, a waiting queue, and policy-controlled active batches. Each request carries an arrival time, prompt-token count, target output-token count, priority weight, and, where required by a policy or metric, an SLO/deadline value. Simulator state includes the pending-arrival list, waiting queue, active requests per GPU, request phase and progress (prefill remaining, decoded tokens, first-token time), current KV-token footprint, per-step active-batch and utilization histories, completed and dropped requests, and observational mechanism telemetry (§4.4).

Time advances in fixed serving steps. At each step, all requests with arrival time less than or equal to the current simulation time are enqueued; the simulator exposes an observable state to the scheduling policy; the policy returns an action specifying admissions and, for policies that support them, preemption, swapping, migration, held decodes, or prefill-control decisions; the simulator validates the action against the resource model and applies it; active requests then advance by one service step; completed requests are removed and recorded; and per-step utilization, batch, KV, and contention histories are updated. The primary benchmark uses the colocated single-GPU-compatible stratum of this model; disaggregated or cross-instance semantics exist for secondary policies but are not pooled with the PRIMARY leaderboard (§4.1).

The resource model includes active-sequence capacity, batch/token-budget constraints where a faithful policy uses them, and KV-memory accounting. KV occupancy is tracked as demand relative to the configured nominal `max_kv_tokens`; only KV-aware policies enforce this limit at admission, so occupancy above 1.0 is interpreted as demand exceeding nominal capacity rather than as an instrumentation failure. In the default backward-compatible mode, prefill is instantaneous and each active decoding request produces one output token per step. In the prefill-aware mode used by faithful serving-style policies, requests may first consume prefill budget before decoding begins; decoding requests produce one token per step; chunked-prefill policies may limit the prefill work admitted into a step; and prefill/decode contention and token-budget saturation are recorded as diagnostics. Admission-control and SLO-aware policies may decline otherwise waiting requests or hold decode work according to their policy semantics; SLO/deadline outcomes are then evaluated through the metric contract (§3.2).

Simulation terminates when all arrivals have been processed and no request remains waiting, active, migrating, or relocating, or when the configured post-arrival drain limit is reached. Requests left in the waiting or transfer states at termination are treated as dropped. Metrics are computed from completed requests, dropped requests, the full arrival population, and the per-step histories; no completion-conditioned metric is imputed when no request completes (§3.2). Given identical requests, configuration, policy, and operating region, the simulator consumes no randomness and produces bit-identical outputs; this was verified by a repeated-run determinism test. This

Table 2. Main simulation-model scope and assumptions used in the PRIMARY ranking-portability study.

| Model component | Treatment in this study |
|---|---|
| Arrivals | Trace-driven request arrivals from frozen 200-request windows. |
| Service progression | Fixed-step serving model; decode emits one token per active decoding request per step. |
| Prefill/decode | Instant prefill in legacy mode; explicit prefill budget and contention diagnostics in prefill-aware modes. |
| Resource limits | Active-sequence, token-budget, and KV-demand constraints as implemented by the configured policy/model. |
| Scheduling decisions | Policies observe simulator state and decide admission, ordering, and supported control actions. |
| Metrics | Computed from arrivals, completions, drops, deadlines, and per-step histories; undefined conditional metrics are excluded, not imputed. |
| Omitted hardware effects | No claim of absolute latency or throughput fidelity to physical GPUs; kernel, networking, allocator, and hardware-noise effects are abstracted in the numerical campaign. |

determinism is what permits the paired design of §3.1: two policies' outcomes on the same window and region differ only because of the policy's own scheduling decisions, not because of independent sampling noise.

### 4.3 Historical fidelity labels and execution limits

The frozen registry distinguishes repository-native classical policies, simulator proxies, style approximations and a class named `FAITHFUL_EXTERNAL`. We retain identifiers such as `slai_faithful` solely as immutable data keys. These names and their historical unit-test gates do not certify native execution equivalence. Policy-specific shadow progress and memory state need not equal the shared execution state. Per-request token intentions are not always carried as exact native runner plans; the resource model is not a native KV allocator.

Source-verified follow-up work found different final-prefill sampling, admission and recomputation semantics in pinned vLLM 0.27.1. Native scheduling can advance optimistic progress before runner acknowledgment; recomputation retains previously emitted output for replay. Historical simulator eviction can instead reset output and first-token progress. These differences are plausible sources of comparative bias, but no measured ranking correction or reversal direction follows from discovering them. We neither repair historical outputs nor relabel them as v2 results.

The numerical campaign remains useful as a controlled comparison of the specified implementations. Determinism verifies repeat execution within that model. It does not establish absolute hardware timing accuracy, mechanism equivalence to production systems, or physical portability of relative rankings.

### 4.4 Mechanism Telemetry

Every cell records seven groups of simulator telemetry: queue depth; active-batch saturation; prefill/decode contention; KV occupancy; admission activations; preemption/reorder events; and token-budget saturation. Means and maxima supplement several groups. A zero can mean that an operation did not occur or that the configured model does not represent that mechanism. These are simulator observations, not GPU counters. The descriptor association analysis in Section 9.4 does not establish a causal explanation of the observed reversals.

Table 3. Frozen panel interpretation. Registry labels are historical identifiers; none of these rows certifies native serving equivalence.

| Policy or group | Historical comparison role |
|---|---|
| `fifo` | Arrival-order baseline and FIFO calibration reference |
| `edf`, `least_laxity_first` | Deadline and slack/urgency ordering |
| `estimated_service_time_first` | Service-length-aware ordering under the simulator's information model |
| `weighted_fair_share` | Priority-weighted fairness discipline |
| `kv_constrained_online` | Simulator KV-pressure proxy |
| `admission_control` | Admission/SLO guard |
| `vllm_faithful` | Historical continuous-batching implementation |
| `vllm_chunked_prefill_faithful` | Historical chunked-prefill implementation |
| `sarathi_faithful` | Historical Sarathi-inspired implementation |
| `slai_faithful` | Historical SLAI-inspired implementation |
| `vllm_style_token_budget`, `scorpio_style_slo_guard` | Two style approximations outside the primary ranking panel |

### 4.5 Implementation identity and comparison scope

A policy label is not a complete treatment definition. The shared simulator service model, admission constraints, policy-local shadow state and input information together determine the observed outcome. When a policy uses request-length information that is available in a trace-driven replay, that information is part of the historical experiment; it must not be represented as an online estimate validated on a production engine. Comparability therefore means controlled replay of the frozen inputs and implementations, not identical internal information or resource enforcement across every named mechanism.

The execution schema records configuration, source, window, policy and repetition identities alongside simulator and manifest hashes. Reproducing a table requires those identities as well as its numerical values. A future corrected simulator or common-native adaptation must receive a separate version and evidence namespace. It cannot replace a policy's old implementation while retaining the old results as if they described the new semantics. That version boundary permits an honest historical comparison without freezing known limitations into future serving work.

## 5 Workloads and Experimental Design

### 5.1 Sources

LSSP's primary benchmark corpus draws from three independently released workload sources: BurstGPT [19], Microsoft's official Azure 2024 LLM-inference trace [10, 16], and Alibaba Cloud's Bailian/Qwen anonymized usage-trace release [3]. Azure's 2023 conversation/code traces [13] and windows from an earlier discriminability pilot (§5.8) contribute additional context but are not part of the primary 120-window matrix beyond the 10-per-source pilot windows reused as described below. TraceLab [25] is retained as an out-of-distribution, coding-agent workload family for future expansion, not as one of the three primary sources, because its independence from a prior 512-window public sweep cannot be fully verified from public artifacts alone.

BurstGPT's own reported collection methodology logs traffic from a regional customer of Microsoft's Azure OpenAI GPT service, and this project's Azure trace is Microsoft's own official

release of internal Azure LLM-inference-service traffic. Both therefore sit on the same underlying cloud provider's infrastructure. We describe BurstGPT and Azure as distinct, independently released workload sources – different collection methodology, different customer population, different release provenance and version history – and we do not describe them as independent providers. Bailian/Qwen (Alibaba Cloud) is a genuinely different underlying platform from both.

### 5.2 Representation and Provenance

Each source is ingested through a dedicated, field-level-audited adapter that maps the source's native schema onto a common internal request representation (arrival time, input tokens, output tokens, total tokens, session identifier where available, model class). Native-field provenance is separate from benchmark synthesis: absent tenant, KV-reuse and failure fields are not inferred, whereas the benchmark supplies priority/SLO parameters for policies and metrics where necessary. These are synthetic experimental attributes, not source-observed service guarantees. License status for each source was verified before any confirmatory use of a real, full-scale sample (for example, BurstGPT is under CC-BY-4.0).

### 5.3 Window Construction

The benchmark corpus freezes exactly 120 windows of 200 requests each: 40 per source (`azure_llm_2024`, `bailian_qwen`, `burstgpt`), each composed of 10 windows reused verbatim from the discriminability pilot (§5.8) plus 30 new windows drawn by a deterministic, outcome-blind, source-symmetric pinned-extension sampler that uses only valid-row index ranges and fixed seeds – never scheduler metrics, policy identities, or the pilot's tie/non-tie outcomes (verified by structural audit to contain no BurstGPT-outcome-targeting signal). Windows are assigned to EARLY/MIDDLE/LATE chronology strata by deterministic relative chronology (60/30/30 windows respectively); the reused pilot windows are all EARLY. The freeze was independently validated – unique window identities, exact per-source and reused/new counts, fixed `request_count = 200`, no prohibited overlap, valid chronology, and absence of any scheduler/policy/load-calibration outcome field in the selection rule – and recorded with a canonical content hash (excluding generation timestamp; full values and verification records are in the preserved artifact hash ledger) before any load calibration or scheduler outcome was generated.

### 5.4 Descriptors

Each window is described by a 28-feature descriptor vector spanning prompt- and output-token-length statistics (mean, p90, long-prompt fraction at the 512/2048/8192-token thresholds), arrival-timing statistics (mean arrival rate, inter-arrival percentiles), and derived load-pressure proxies. Three deterministically redundant pressure-proxy features ($R^2 = 1.0$ against other included features) are retained in the underlying dataset but excluded from the classifier input used in §5.5. Prompt-token figures compose differently across sources for architectural reasons: TraceLab's and Azure's prompt-token fields are potentially multi-turn-cumulative context, while BurstGPT's is a single per-request figure. This is a property of how each source's underlying workload is architected (a growing agent/conversation session versus a single stateless API call), not an adapter artifact, and it is preserved as an explicit caveat on every prompt-length-based claim below.

### 5.5 Characterization scope

A separate descriptor study used 400 windows across four classes, including TraceLab. It is not part of the 120-window outcome matrix. Its classifier results and feature-attribution claims require their own provenance and are not evidence that a particular feature causes ranking reversals. This

draft retains the source and prompt-semantics caveats without using those classifier accuracies as central portability evidence.

### 5.6 Operating-Region Calibration

A FIFO-only calibration pass establishes six operating regions – `LOW` (load factor 0.5), `PRE_KNEE` (0.8), `KNEE` (1.0), `POST_KNEE` (1.1), `OVERLOAD` (1.2), and `HIGH_PRESSURE` (1.5) – across all 120 frozen windows (720 calibration cells total; 720 valid, deterministic, 0 failed/missing/ duplicated). We report this as calibration provenance, not a comparative scheduler result: FIFO is the calibration reference policy, and the calibration's role is to fix load-factor-to-region mappings before any other policy is executed against them, not to characterize FIFO's relative performance. The preserved 720-cell raw FIFO calibration manifest contains no zero-completion cells. Earlier manuscript prose reporting 301 such records is inconsistent with this artifact and is not retained as a finding. The distinction between calibration and campaign configuration is preserved: a calibration record is not an outcome for every policy under the final campaign.

### 5.7 Paired Design and Repetitions

Because the simulator is fully deterministic (§4.2), a single execution per (window, policy, region) cell is sufficient: each of the 9,360 configurations yields one scientific outcome, and its rep0/rep1 pair (18,720 executions in total) is a deterministic verification pass confirming bit-identical output, not an independent stochastic replicate. Ranking-uncertainty quantification (§7) instead comes from resampling over the workload windows themselves (block bootstrap), not from repeated stochastic runs of the same cell.

### 5.8 Protocol freeze, pilot and overlap

The repository preserves the workload, panel, calibration, campaign and analysis freezes and their hashes. The earlier discriminability pilot motivated the larger panel; it is not confirmatory ranking evidence. The principal outcome matrix is fixed. Subsequent metric and SLO analyses are labeled extensions, with their original configurations unchanged. Reused pilot windows are disclosed rather than counted as an independent replication of the pilot. Historical source and freeze identities are retained even where later prose or engineering audits qualify their meaning.

A related load-scaling manuscript and a separate module-intervention project share research lineage. This paper does not claim their results or intervention-based attribution as its own. The earlier 60-window project's exact byte-level overlap is not fully reconstructed; final overlap review remains necessary. The present sampler is distinct, but that alone is not a proof of zero overlap.

### 5.9 Window and trace eligibility

Every principal window contains 200 requests; a sweep of requests per window has not been performed. The recovery analysis varies the number of windows, not requests per window. Source-reported token counts are not retokenized model inputs. All 40 Azure and 40 Bailian window medians exceed 256 source tokens. The later whole-prefill engineering contract, bounded at 256 prompt tokens and 16 active sequences, therefore does not cover these traces. No requests were truncated or filtered for this manuscript preparation, and no mapping to a modern model/tokenizer context limit is silently assumed.

### 5.10 Temporal partitions

The manifest assigns 20 EARLY, 10 MIDDLE and 10 LATE windows per source (60/30/30 over all three sources). Thus 60/30/30 is not a BurstGPT-only count. BurstGPT uses relative-chronology terciles and an additional bisect sensitivity split. Azure uses the frozen 2024-05-15 calendar boundary;

Table 4. Range of window-level median source-reported prompts in the frozen 40-window/source corpus. These are not common-tokenizer counts.

| Source | Minimum median | Maximum median | Medians above 256 |
|---|---|---|---|
| azure_llm_2024 | 628.5 | 1595 | 40/40 |
| bailian_qwen | 343.5 | 1150 | 40/40 |
| burstgpt | 140.5 | 620.5 | 9/40 |

Table 5. Primary ANWG cross-source comparison records by region. Each row contains 55 policy pairs times three source pairs; these are overlapping comparisons, not independent experimental units.

| Region | Stable | Microscopic | Supported | Total |
|---|---|---|---|---|
| LOW | 161 | 4 | 0 | 165 |
| PRE_KNEE | 149 | 10 | 6 | 165 |
| KNEE | 149 | 10 | 6 | 165 |
| POST_KNEE | 147 | 12 | 6 | 165 |
| OVERLOAD | 147 | 8 | 10 | 165 |
| HIGH_PRESSURE | 145 | 12 | 8 | 165 |

Bailian is labeled relative chronology only. These definitions remain unchanged and are interpreted separately.

### 5.11 Recorded prompt-length coverage

These summaries demonstrate a coverage gap for bounded short-prompt serving checks, not an exact request-exclusion fraction. A window median does not identify every request above a threshold, and source length fields do not recover absent prompt text. No benchmark text is synthesized here to claim tokenizer equivalence. Model, tokenizer and context eligibility for prospective native traces require separate decisions and measurements.

## 6 Performance Within Operating Regions

The operating-region names identify FIFO-referenced arrival-rate settings, not independently measured physical saturation states. Each source has 40 windows and every primary policy is observed on each window in each region. Comparisons within a source/region use matched windows; a policy is not calibrated separately to obtain a favorable operating point.

The central quantity is relative order under ANWG, the fraction of arrival priority weight receiving deadline credit. Completion-conditioned latency and throughput can describe a different subset of requests. A policy that finishes only favorable requests can have a strong completion-conditioned metric while losing arrival-normalized utility; the common denominator and missing-value rules are therefore part of the comparison, not merely reporting details.

Across the complete two-pass campaign, 211 of 9,360 unique configurations have zero completions; 48 of these belong to the 11-policy primary panel. The preserved FIFO calibration manifest instead has zero zero-completion records out of 720. Calibration records cannot be propagated as zero-completion outcomes for all other policies. The reported rankings retain defined policy/window observations and omit undefined conditional observations, so effective observation sets can differ even when a condition-level ranking remains estimable.

Table 5 separates small ordering changes from supported reversals. No primary record is unestimable or a practical sign change with a wide saved interval. This does not prove that every microscopic sign change is statistically insignificant: the frozen classification stops at the margin test and need not evaluate its interval. The absence of supported reversals at LOW is an observed result under the fixed metric and margin, not a reason to exclude that region.

The full per-policy outcome matrix remains the record for within-region performance. This paper does not turn the panel into a universal winner recommendation: the policies share a simulator implementation boundary, and region names, synthesized deadlines and metric conditioning all constrain the interpretation of an ordering.

## 7 Portability Across Workloads and Regions

### 7.1 Cross-Source Rankings

Across the 3 source pairs × 6 operating regions (18 conditions), Kendall's tau-b for the PRIMARY metric ranges from 0.55 to 1.00, with a per-region mean between 0.69 and 0.77 (Figure 1). Every one of the 18 conditions has a positive, non-trivial tau-b: comparative scheduler rankings are never uncorrelated or inverted *in aggregate* between any two sources at any region. Portability is, however, visibly uneven across source *pairs* rather than uniform: Azure-2024 and Bailian/Qwen agree very strongly with each other at every region ($\tau = 0.89$–$1.00$), while every comparison involving BurstGPT is substantially lower ($\tau = 0.55$–$0.71$) – a gap of roughly 0.2–0.4 in Kendall's tau-b that holds at all six regions, not just one. Top-1 agreement (the deterministic single top-choice policy) and top-3 agreement follow the same source-pair pattern in the underlying evaluation data; we do not tabulate top-1/top-3 separately in the main text to avoid restating the same qualitative pattern three times, but the full table is part of the preserved artifact (§11).

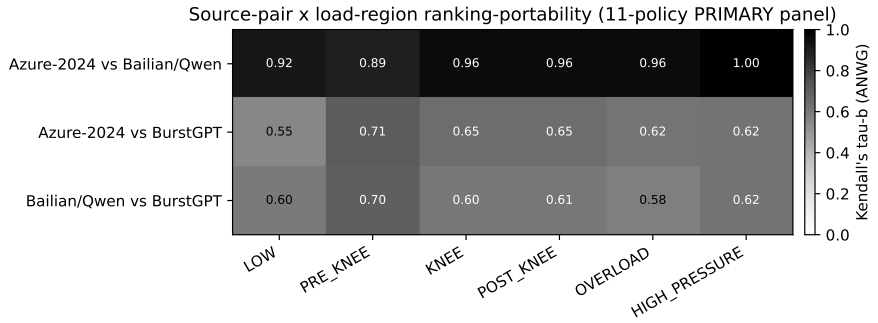



Fig. 1. Source-pair × load-region ranking-portability: Kendall's tau-b between each pair of sources' PRIMARY-panel (11-policy) rankings on the ANWG metric, at each of the six frozen operating regions (§5.6). Computed from the frozen ranking correlations artifact.

The min/max range in Figure 1 makes clear that portability is not homogeneous even within a region: the range at LOW (0.55–0.92) is nearly as wide as at any other region, and the range is driven entirely by which source pair is compared, not by which region is compared (§7.2 tests region dependence explicitly). No region shows uniformly low or uniformly high agreement across all three source pairs simultaneously.

### 7.2 Load Conditioning

Load-level dependence is a secondary robustness/sensitivity analysis (§3), reported here as a direct read of the same §7.1 table rather than a separate headline claim of cross-source rank instability by itself. Secondary ranking statistics (Spearman's $\rho$ and top-1/top-3 overlap) exhibit an identical quantitative pattern and are omitted from the main tables for concision; their full values are preserved in the preserved analysis outputs. Figure 1's per-region mean tau-b is close to flat across the six regions (0.69 at LOW, rising to 0.72–0.77 elsewhere, no monotonic trend with load factor); a leave-one-region-out comparison between the full six-region grid and the four-region LOW/PRE_KNEE/KNEE/ OVERLOAD subset used by LOAD_CALIBRATION_SENSITIVITY (§9) changes the mean tau-b by less than 0.01 (0.728 on the four-region subset versus 0.733 on the full six-region grid) – the headline cross-source portability conclusion is not sensitive to the choice of four-region versus six-region calibration grid. Load regime does, however, materially affect *where practically supported reversals occur* (§7.3): reversal counts are concentrated at PRE_KNEE and above, essentially absent at LOW.

### 7.3 Pairwise Reversals

Of the 990 (policy pair) × (source pair, region) PRIMARY-metric records (55 policy pairs among the 11 PRIMARY policies × 18 conditions), the five-class breakdown (§3.3) is: 906 STABLE_NO_SIGN_CHANGE (91.5%), 48 MICROSCOPIC_SIGN_CHANGE (4.8%), 36 SUPPORTED_PRACTICAL_REVERSAL (3.6%), and zero records in either UNDEFINED_UNESTIMABLE or UNSUPPORTED_SIGN_CHANGE_WIDE_CI. The large majority of pairwise comparisons are stable; a small but non-trivial minority (about 1 in 28) are both practically and statistically supported reversals – never a majority, and never absent. The saved taxonomy contains no practical sign change with a wide interval. Microscopic changes are tested before the interval branch, so this is not evidence that practical and statistical criteria always agree.

Reversal counts are load-dependent: zero SUPPORTED_PRACTICAL_REVERSAL records at LOW, rising to 6 at each of PRE_KNEE, KNEE, and POST_KNEE, 10 at OVERLOAD, and 8 at HIGH_PRESSURE (Table 5). All 36 involve BurstGPT, but they span five policy pairs: SLAI–vLLM (10), Sarathi–SLAI (9), chunked-vLLM–SLAI (9), Sarathi–vLLM (4), and chunked-vLLM–vLLM (4), where these names abbreviate historical simulator identifiers. Earlier manuscript wording attributing all 36 records to one pair was incorrect. The single largest-effect-size supported reversal (operationalized as the smaller-magnitude of the two conditions' practical margins, the binding constraint under the frozen practical-margin rule, §3.3) is `slai_faithful` vs. `vllm_faithful` at HIGH_PRESSURE, comparing Azure-2024 (or, at an effectively tied effect size, Bailian/Qwen) against BurstGPT – this is the case selected for the historical physical comparison under the frozen selection rule (§10.2).

A representative stable comparison, used as a null-result control alongside the reversal case in the historical physical design (§10.2), is the source pair with both the highest tau-b and (among the highest-tau conditions) the narrowest bootstrap interval: Azure-2024 vs. Bailian/Qwen at HIGH_PRESSURE ($\tau = 1.00$, 95% CI $[0.90, 1.00]$).

### 7.4 Temporal Portability

Following the frozen temporal design (§5.10), four temporal record types are reported separately, each under its source's own chronology, and never merged into a single "temporal/domain OOD" number: BurstGPT's EARLY/MIDDLE/LATE tercile split (primary) and EARLY/LATE bisect (sensitivity check on the tercile boundary), Azure-2024's absolute calendar split, and Bailian/Qwen's RELATIVE_CHRONOLOGY_ONLY comparison, for which no calendar-based claim is made. All values are keyed identically to the source temporal robustness artifact. The canonical results reveal high

Table 6. All 18 primary source-pair comparisons. A=Azure, B=Bailian, G=BurstGPT. Intervals are the preserved 95% window-bootstrap intervals; M1 independently reproduced the point estimates but did not rerun intervals.

| Region | Pair | Tau-b | Saved 95% interval |
|---|---|---|---|
| LOW | A–B | 0.920 | [0.864, 0.920] |
| LOW | A–G | 0.551 | [0.203, 0.551] |
| LOW | B–G | 0.599 | [0.220, 0.599] |
| PRE_KNEE | A–B | 0.893 | [0.893, 1.000] |
| PRE_KNEE | A–G | 0.709 | [0.450, 0.753] |
| PRE_KNEE | B–G | 0.697 | [0.404, 0.753] |
| KNEE | A–B | 0.958 | [0.914, 1.000] |
| KNEE | A–G | 0.646 | [0.578, 0.729] |
| KNEE | B–G | 0.604 | [0.521, 0.688] |
| POST_KNEE | A–B | 0.958 | [0.958, 1.000] |
| POST_KNEE | A–G | 0.647 | [0.510, 0.678] |
| POST_KNEE | B–G | 0.606 | [0.469, 0.647] |
| OVERLOAD | A–B | 0.958 | [0.958, 1.000] |
| OVERLOAD | A–G | 0.620 | [0.566, 0.620] |
| OVERLOAD | B–G | 0.580 | [0.525, 0.620] |
| HIGH_PRESSURE | A–B | 1.000 | [0.904, 1.000] |
| HIGH_PRESSURE | A–G | 0.623 | [0.550, 0.712] |
| HIGH_PRESSURE | B–G | 0.623 | [0.550, 0.699] |

rank stability across time: BurstGPT's tercile split shows strong agreement between EARLY and LATE chunks across non-LOW load regions ($\tau_b \in [0.81, 1.0]$), while the BISECT sensitivity check yields similarly stable correlations ($\tau_b \in [0.72, 0.80]$). At LOW, one BurstGPT tercile comparison instead has $\tau_b = -0.10$; this exception must not be hidden by a non-LOW summary. Azure's calendar split produces near-perfect correlation ($\tau_b \in [0.90, 1.0]$), and Bailian's relative-chronology split likewise exhibits minimal rank change ($\tau_b \in [0.94, 1.0]$).

Although the frozen protocol for this research question specified an analysis of temporal, provider, and domain shifts, a standalone domain-shift component was not separately operationalized in the final artifacts. The evidence reported here addresses the temporal component directly. The frozen cross-source comparisons (§7.1) include the one genuine provider boundary in this corpus (Bailian/Qwen on Alibaba Cloud versus the two Microsoft-infrastructure sources, §5.1), but the design does not isolate a causal provider effect from ordinary cross-source variation – BurstGPT-vs-Azure is a cross-source, not cross-provider, comparison; domain shift, like the provider component, is addressed only implicitly through these same cross-source comparisons and was not separately operationalized.

### 7.5 Point estimates and saved uncertainty

The intervals describe a window-resampling procedure on the sampled sources, not uncertainty over hardware architectures or all production populations. Their width is distinct from between-source variation in point estimates. Reporting them next to the full comparison set avoids selecting only the strongest or weakest agreement. Stable source pairs and the absence of a supported reversal at LOW remain part of the result.

## 8 Portability Across Metrics

§7.3 and §9.1 ask whether a policy ranking under one fixed metric (ANWG) is portable across sources, regions, and windows. A different question – whether a ranking is portable across *metrics*, for the same source and region – is addressed here as an explicitly labeled post-campaign extension that reuses the primary analysis's own statistical methods without modifying, re-running, or re-deriving the underlying outcome matrix. Across all 3 sources × 6 regions × 55 metric pairs among the study's tracked outcome metrics (990 conditions), rank correlation between metrics is on average moderate (median Kendall's $\tau_b = 0.419$, range $[-0.506, 1.000]$; 202/990, 20%, of conditions have *negative* $\tau_b$), and the single top-ranked policy agrees between the two metrics in only 316/990 (31.9%) of conditions (disagreeing in the remaining 68.1%). At the level of individual pairwise policy comparisons (54,450 total, one per (metric pair, condition, policy pair), generated from the 55 unique pairs among the 11 eligible tracked metrics: ANWG, completion fraction, weighted completion fraction, SLO-violation rate, weighted goodput, mean latency, p95 latency, request throughput, token throughput, mean TTFT, and p95 TTFT; p99 latency was ineligible and excluded), the same five-class taxonomy as §7.3 (ordinary, not FDR-adjusted, bootstrap-CI classification; §3.3) finds 3,244 (6.0%) show a statistically supported practical disagreement in which policy is preferred depending on which metric is used, versus 45,800 (84.1%) in the same order, 5,205 (9.6%) a sign change too small to be practically meaningful, and 201 (0.4%) an unsupported sign change with a wide confidence interval; no aggregate comparison in this policy panel was undefined or unestimable; individual policy/window observations can nevertheless be missing. Unlike §7.3's primary reversal check, the separate BH-FDR correction removes some of these cross-metric disagreements: of the 3,244 raw-CI-supported records, 3,207 (98.9%) also survive the within-family BH-FDR check, while 37 (1.1%) do not, meaning the multiplicity-corrected count of cross-metric disagreements is 3,207 rather than 3,244 – a genuine, disclosed sensitivity to the multiplicity correction, not an error in either figure. Scheduler-ranking portability is therefore not only source- and region-dependent (§7); which metric is used to define "better" materially changes the ranking in a non-trivial minority of conditions, and the median condition shows no default expectation that the two metrics' top choices should agree.

### 8.1 What a preferred-policy change means

All 55 unordered pairs among 11 metrics are included at each of three sources and six regions. Thus 990 denotes metric-pair conditions here; it denotes policy-pair/source-pair/region records in the primary reversal analysis. These are different denominators even though their sizes match. The 54,450 records further expand each metric-pair condition into 55 policy-pair comparisons. They share windows, policies and metrics, so they must not be interpreted as 54,450 independent replications.

Lower-is-better metrics are direction-normalized before comparison. Kendall tau-b accommodates equal aggregate values; top-1 uses a stable policy-ID tie break. Among the 1,980 metric sides of the 990 records, 1,010 have more than one policy tied at the maximum. The 31.9% top-choice agreement is exactly reproducible under the historical rule but should not be described as disagreement between unique optimal policies. A set-valued optimum-overlap analysis would be a separately specified extension; it is not retroactively substituted for the reported statistic.

The median correlation describes this finite set of metric comparisons, not a sampling distribution over all conceivable service objectives. Correlated metrics, conditional observation sets and metric-specific priority weighting can explain some disagreement by construction. The analysis does not identify a uniquely correct deployment metric. It requires a reader to state the intended objective before transporting a ranking. Its post-campaign status limits confirmatory interpretation.

## 9 Robustness and Practical Implications

### 9.1 Robustness

Of the seven preregistered robustness families (§9), six were computed (five in the ranking-correlation, temporal-robustness, and sample-complexity outputs below, and a sixth, `LEAVE_ONE_POLICY_FAMILY_OUT`, in the telemetry-explanation analysis of §9.4). The seventh, `METRIC_DEFINITION_SENSITIVITY`, was never implemented; we disclose this as a protocol gap rather than perform a post hoc analysis after the results were already known.

- **`PRIMARY_ONLY`.** Identical to the headline result by construction: the ranking correlations artifact already restricts every comparison to the 11-policy PRIMARY panel (§7.1), never mixing in the two `STYLE_APPROXIMATION` robustness policies.
- **`LEAVE_ONE_SOURCE_OUT`.** Excluding BurstGPT leaves only the Azure-2024/Bailian-Qwen pair, whose mean tau-b is 0.948 – close to the ceiling. Excluding Azure-2024 or Bailian/Qwen each leaves a BurstGPT-involving pair, with mean tau-b 0.618 and 0.633 respectively. This is the same asymmetry as §7.1, now stated per excluded source: the headline portability figure is high specifically because two of the three sources agree strongly, and is pulled down specifically by BurstGPT, not uniformly by all three sources.
- **`LOAD_CALIBRATION_SENSITIVITY`.** Reported in §7.2: mean tau-b changes by less than 0.01 between the four- and six-region grids.
- **`TEMPORAL_SPLIT_SENSITIVITY`.** The BurstGPT bisect-vs-tercile comparison from §7.4; both split definitions are reported side by side in the preserved table rather than collapsed into one number.
- **`WINDOW_SIZE_SENSITIVITY`.** Despite its name, this item is the window-*count* subsampling ladder of §9.2, not a requests-per-window sweep (§5.9), and is not separately recomputed here.
- **`LEAVE_ONE_POLICY_FAMILY_OUT`.** Excluding any single one of the ten frozen mechanism families changes the headline mean tau-b only modestly, from 0.707 (excluding the classical arrival-order family) to 0.792 (excluding the SLO-aware family), against the full-panel baseline of 0.733. No single mechanism family's removal collapses or inflates the headline portability figure – the source-pair-driven pattern of §7.1 is not an artifact of any one policy family dominating the panel.

### 9.2 Finite-Corpus Ranking Recovery

*Scope.* The reference is the frozen 40-window ranking, not an unknown population optimum. Recovery is conditional on this finite corpus and the historical tie rule.

*Design and results.* The 0.9 recovery probability used throughout this section is a pre-registered benchmark reporting target, fixed before any sample-complexity output was computed: it provides one interpretable, high-recovery criterion for comparing sources and metrics against each other, and is not asserted to be a universal statistical-significance level or a field-standard adequacy cutoff for scheduler benchmarking.

For the PRIMARY metric, recovery of the exact $n = 40$ reference ranking is slow and strongly source-dependent (Figure 2): BurstGPT never reaches the preregistered 0.9 exact-order recovery threshold below the full window budget (0.2% at $n = 5$, rising only to 56.6% at $n = 30$), Bailian/Qwen similarly does not cross 0.9 before the $n = 40$ ceiling (64.0% at $n = 30$), while Azure-2024 crosses the threshold already at $n = 30$ (98.8%). None of the three sources reaches 0.9 exact-order recovery at $n \in \{5, 10, 20\}$. Because $n = 40$ is the size of the full reference pool itself, exact recovery at $n = 40$ is 1.0 by construction (sampling all 40 windows without replacement reproduces the reference

exactly) and is not an informative data point on its own; the informative content is in how slowly each source's curve rises toward that trivial ceiling.

Top-1 recovery (does the resampled subset identify the single best-performing PRIMARY-panel policy correctly) is a materially easier question than exact-order recovery for two of the three sources: Azure-2024 and Bailian/Qwen both exceed 0.9 top-1 recovery already at $n = 5$ (96.6% and essentially 100% respectively), even though their *exact*-order recovery at $n = 5$ is only 35.0% and 14.6%. For these two sources, identifying the single best policy requires far less evidence than reconstructing the complete 11-policy ordering. BurstGPT shows no such gap: its top-1 recovery is comparably slow to its exact-order recovery (13.0% at $n = 5$, 74.2% at $n = 30$), remaining below 0.9 until the trivial $n = 40$ ceiling – consistent with §7.1's finding that BurstGPT is the source whose relative ordering is hardest to pin down. Comparing a concentrated (single-source-heavy) against a distributed (evenly-spread-across-sources) window budget of equal total size, the distributed budget achieves higher mean Kendall's tau-b against the reference ranking (0.986) than the concentrated budget (0.940) for the primary metric – spreading a fixed evaluation budget across sources recovers the reference ranking more reliably than concentrating it in one source.

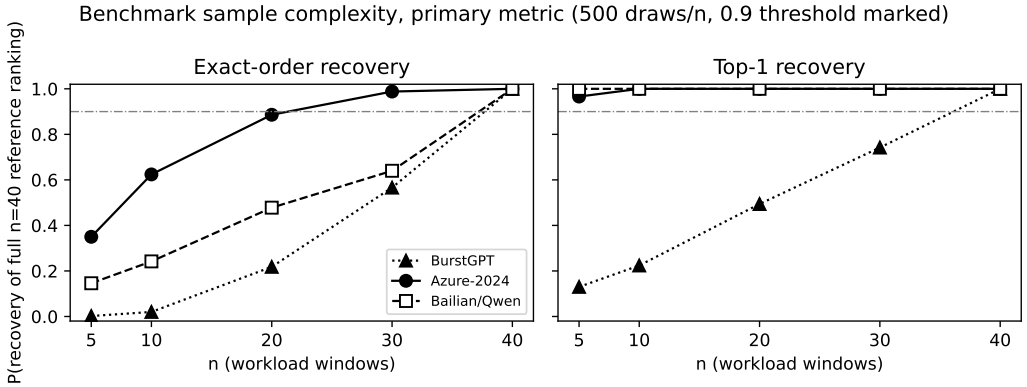



Fig. 2. Benchmark sample-complexity recovery curves, PRIMARY metric, 500 draws per $n$, sampling without replacement. Left: exact-order recovery of the full $n = 40$ reference ranking. Right: top-1 recovery. The 0.9 threshold is marked. Computed from the frozen sample complexity artifact.

This analysis directly answers RQ5 (§3) and depends on the full primary outcome matrix (§7); it is not run on the earlier, smaller discriminability pilot (§5.8).

### 9.3 SLO-Definition Sensitivity

As disclosed in §9.4, this check was never part of the preregistered protocol; we report it here as the post-campaign extension it was actually run as: three alternative SLO-synthesis multipliers (`tight_10x`, `primary_20x` – this study's own convention – and `loose_40x`) applied across the 5 load regions that contain at least one PRIMARY-metric supported reversal (excluding `LOW`, which accounts for 0 of the 36 supported reversals and so has nothing for a sensitivity check to stress-test), evaluating 11 policies × 120 total windows × 5 regions × 3 variants (19,800 input rows according to the preserved extension status), with every seed held identical to §7.

**Ranking robustness.** Across the 30 (source, region, variant) conditions compared against the study's primary SLO convention, rank correlation stays high under either alternative (median Kendall's $\tau_b = 0.872$, range $[0.533, 1.000]$); the single top-ranked policy nonetheless changes under an alternative SLO definition in 13/30 (43%) of conditions.

**Reversal persistence.** We check the 36 PRIMARY-metric supported reversals from §7.3 against the alternative definitions. Because there are 3 SLO variants (tight, primary, loose), there are $36 \times 3 = 108$ mathematical comparisons in the artifact's persistence matrix, of which 36 are identity checks against the primary definition itself (all 36 trivially persist). Restricting focus to the 72 scientifically meaningful non-trivial comparisons against the alternative definitions: 52/72 (72%) persist, 6/72 (8%) change direction, 10/72 (14%) remain the same direction but fall below the practical-margin threshold under the alternative definition, and 4/72 (6%) become statistically indistinguishable from zero. The six direction changes contradict a claim of invariant reversal direction under alternative deadlines. A non-trivial minority (20/72, 28%) are sensitive to the SLO-definition choice – disclosed here as a caveat on those specific reversals, not grounds to discard them.

### 9.4 Explanations and unresolved questions

The saved descriptor regression has 42 usable fits among 990 records; no standard errors establish a directional explanatory claim. Source identity, prompt structure and simulator execution semantics remain confounded. The descriptive association is not a causal mechanism result or a deployment selector. The synthetic-to-real pilot is engineering context only; it does not answer RQ3 at the level of the primary corpus.

The SLO extension was specified after the primary outcomes were known. The alternative-metric-definition robustness item remains distinct from that deadline-synthesis extension. Neither replaces a missing prospective analysis, and requests-per-window sensitivity is still unmeasured.

Practically, a ranking should travel with its implementation identity, workload/window definition, calibration, metric denominator, tie rule and uncertainty procedure. Agreement on that record is evidence for the specific comparison. A changed objective or execution model warrants reassessment; it does not license assuming a reversal. No held-out portability predictor or cost-effective recalibration rule is validated here.

## 10 Threats to Validity

### 10.1 Simulator validity and policy comparability

The primary findings are conditional on a project-local, deterministic, fixed-step simulator. Native partial-prefill execution, allocation refusal, KV preemption and recomputation are not reproduced merely by a policy's name or unit-test success. Historical policies can maintain shadow progress and KV tables that differ from shared execution state. Some resource limits are policy-dependent; demand above nominal KV capacity is possible. The simulator is therefore a comparison of these implemented abstractions, not a controlled physical comparison isolating only production scheduler selection rules. Differences in abstraction can influence rankings.

Later source audits reproduced discrepancies in final-prefill sampling, output-prefix retention under recomputation and admission accounting. Native scheduling counters are not execution acknowledgments. Intermediate prefill can produce no sample, while final prefill can produce the first output token. Restarting generated progress changes work and first-token accounting. No correction factor translates historical simulated timing to physical timing; no numerical ranking effect was measured by those audits.

### 10.2 Physical evidence boundary

The legacy RQ6 artifact records 240 physical executions of two configured policy paths across three sources and 40 windows per source. A retrospective hardware audit established 10 GB A100 MIG allocations, with 9.5 GiB visible to CUDA, rather than full 80 GB devices. Parent-card names were

not allocated-device identity evidence. Individual allocated UUIDs and exact compute-slice profiles were not preserved.

More seriously, request class metadata was not transported to the legacy SLAI plugin; missing classes selected fallback behavior. Its deadline clock used a synthetic iteration-to-time conversion. Client worker limits and prompt construction could delay actual arrivals, which were not fully instrumented, and adapter plans could differ from native scheduled work. The saved numerical observations remain historical evidence of that runner. They do not establish faithful SLAI-versus-native ranking agreement or reversal non-reproduction. The mismatch cannot be attributed uniquely to simulator error. RQ6 remains unresolved at the intended fidelity level.

A later full-A100 engineering smoke established bounded native/checked-SLAI serving on short synthetic fixtures. It is separate from this campaign and does not validate realistic-trace ranking portability. Subsequent CPU ledger and native-boundary conformance work uses synthetic execution replies; it is not GPU timing, numerical-output or performance evidence. The P2B control hook rejects selected waiting work preceding selected resident work in one batch and has no atomic native-state rollback on late allocation failure. General partial-prefill serving, KV pressure/preemption, recomputation and distributed TP2/DP2 remain separate gates. Nothing in this manuscript establishes cross-architecture or parallelism portability.

### 10.3 Statistical and construct validity

The 18,720 executions comprise 9,360 deterministic pairs, not stochastic replications. Window bootstrap intervals characterize variation within the sampled trace corpus under a window-exchangeability assumption. Production traces can retain temporal or session dependence across windows. Independent publication of traces is not independent sampling of providers; BurstGPT and Azure share infrastructure lineage. Provider and domain effects are not causally isolated. The temporal definitions differ by source and should not be pooled into a common domain-shift statistic.

Missing completion-conditioned observations are excluded, not imputed. An estimable aggregate does not imply that every policy/window observation exists. In the primary panel, 48 unique configurations have zero completions; the preserved calibration manifest has none. Earlier prose reporting 301 calibration zero-completion cells is disputed by that file. Survivorship conditioning, deterministic tie breaks and synthetic deadlines affect the meaning of a preferred policy. Practical-margin and recovery thresholds are reporting conventions, not universal systems standards. The raw-CI reversal taxonomy and the additional within-family BH-FDR flag answer different questions; neither is a family-wise error guarantee. No global multiplicity claim across all analyses is made.

The cross-metric and SLO extensions were conducted after primary results were available. Their frozen extension configurations support reproducibility, not retrospective preregistration. The descriptor regressions are sparse and descriptive. The finite-reference recovery curves do not prove that 40 windows suffice for a new source or that a full-reference ordering is a population truth. Alternative requests-per-window sizes, held-out sources and a prospective transfer predictor are not evaluated.

### 10.4 Coverage, provenance and reproducibility

Source token counts may represent different context semantics and are not necessarily counts under a common tokenizer. Long prompts in the Azure and Bailian windows are outside the later 256-token engineering contract. No retokenization or context-eligibility audit can be inferred from the short smoke. The 120 selected windows are a finite corpus, not a representative sample of every production LLM service.

Historical source, campaign and analysis files are preserved with hashes. M1 independently reconciles execution counts, numerical pairs, primary Kendall point estimates, stored reversal

classifications and cross-metric point estimates. It does not regenerate every bootstrap interval, establish fresh hardware reproducibility, or retroactively validate source licensing and public release accessibility. Bibliography metadata and the prior manuscript overlap boundary require final author review. Unresolved checks are listed explicitly in the accompanying claim-to-evidence register.

## 11 Conclusion

LSSP treats the portability of a comparative scheduler ranking as an explicit evaluation output. Its historical simulator corpus shows strong but uneven cross-source agreement, a minority of practical reversals spanning five policy pairs, and more limited agreement across metrics. The contribution is the reproducible comparison design and the conditional empirical characterization, not a novel rank statistic or a universal scheduler recommendation.

These findings apply to the frozen implementations, windows, regions and metric rules. Later audits narrow the native-system fidelity claims: the legacy physical comparison cannot establish faithful scheduler-ranking transfer, and CPU conformance is not physical execution evidence. Future serving measurements require separately validated admission, execution, KV, timing and output contracts before testing portability on hardware.

## Artifact availability and declarations

The review repository preserves the source, configurations, frozen outcome matrix, canonical analyses and the M1 claim audit. The dataset package `dataset/v1.1.0` includes per-execution outcomes, analysis JSON and reproducible tabular views; it does not include a full dump of the legacy physical request records. Historical Hugging Face and Zenodo release references require a current accessibility and version check before submission. The working repository is private; this draft makes no claim that its new review files are publicly available. Upstream trace redistribution remains subject to the original terms.

The author declares no competing interests. Historical acknowledgments identify support from Google Cloud Research Credits, the CloudRift AI Builder Grant and the Cohere Labs Catalyst Grant Program, and acknowledge Professor Ioannis Koutis and NJIT's Wulver/ARCS resources. Funding and contribution statements require author confirmation for the final submission. The author is responsible for conceptualization, methodology, software, validation, analysis, data curation, visualization and writing. AI-assisted preparation was used historically and Codex assisted this M1 revision; the revised claims and text remain pending detailed author review and approval.

## References

[1] Amey Agrawal, Nitin Kedia, Jayashree Mohan, Ashish Panwar, Nipun Kwatra, Bhargav S. Gulavani, Ramachandran Ramjee, and Alexey Tumanov. 2024. VIDUR: A Large-Scale Simulation Framework for LLM Inference. In *Proceedings of Machine Learning and Systems (MLSys '24)*. MLSys. https://arxiv.org/abs/2405.05465

[2] Amey Agrawal, Nitin Kedia, Ashish Panwar, Jayashree Mohan, Nipun Kwatra, Bhargav S. Gulavani, Alexey Tumanov, and Ramachandran Ramjee. 2024. Taming Throughput-Latency Tradeoff in LLM Inference with Sarathi-Serve. In *Proceedings of the 18th USENIX Symposium on Operating Systems Design and Implementation (OSDI '24)*. USENIX Association. https://www.usenix.org/conference/osdi24/presentation/agrawal

[3] Alibaba Cloud. 2026. *qwen-bailian-usagetraces-anon.* https://github.com/alibaba-edu/qwen-bailian-usagetraces-anon/tree/5f7439c51ec248a0c585f7d90a41a6f57773b912 Anonymized Bailian/Qwen production LLM-serving usage-trace release; repository `alibaba-edu/qwen-bailian-usagetraces-anon`, pinned commit `5f7439c51ec248a0c585f7d90a41a6f57773b912`, Apache-2.0.

[4] Agrim Bari, Parikshit Hegde, and Gustavo de Veciana. 2025. Optimal Scheduling Algorithms for LLM Inference: Theory and Practice. *Proceedings of the ACM on Measurement and Analysis of Computing Systems* 9, 3 (2025), 59. doi:10.1145/3771574

[5] Jaehong Cho, Hyunmin Choi, and Jongse Park. 2025. LLMServingSim2.0: A Unified Simulator for Heterogeneous Hardware and Serving Techniques in LLM Infrastructure. *IEEE Computer Architecture Letters* 24, 2 (2025). https:

//arxiv.org/abs/2511.07229

[6] Jaehong Cho, Minsu Kim, Hyunmin Choi, Guseul Heo, and Jongse Park. 2024. LLMServingSim: A HW/SW Co-Simulation Infrastructure for LLM Inference Serving at Scale. In *Proceedings of the 2024 IEEE International Symposium on Workload Characterization (IISWC '24)*. IEEE. https://arxiv.org/abs/2408.05499

[7] Yichao Fu, Siqi Zhu, Runlong Su, Aurick Qiao, Ion Stoica, and Hao Zhang. 2024. Efficient LLM Scheduling by Learning to Rank. In *Proceedings of the 38th Conference on Neural Information Processing Systems (NeurIPS '24)*. https://arxiv.org/abs/2408.15792

[8] Yunho Jin, Chun-Feng Wu, David Brooks, and Gu-Yeon Wei. 2023. $S^3$: Increasing GPU Utilization during Generative Inference for Higher Throughput. In *Advances in Neural Information Processing Systems 36 (NeurIPS '23)*. doi:10.5555/3666122.3666913

[9] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. 2023. Efficient Memory Management for Large Language Model Serving with PagedAttention. In *Proceedings of the 29th Symposium on Operating Systems Principles (SOSP '23)*. ACM. doi:10.1145/3600006.3613165

[10] Microsoft Azure. 2024. *AzureLLMInferenceDataset2024*. https://github.com/Azure/AzurePublicDataset/blob/master/AzureLLMInferenceDataset2024.md

[11] William Nixon, Jon Durbin, Florian Standhartinger, Haryadi S. Gunawi, and Juncheng Yang. 2026. A Year in LLM Serving: Workload Evolution, Caching and Load-Balancing. *arXiv preprint* (2026). https://arxiv.org/abs/2608.13573

[12] Konstantinos Papaioannou and Thaleia Dimitra Doudali. 2024. The Importance of Workload Choice in Evaluating LLM Inference Systems. In *Proceedings of the 4th Workshop on Machine Learning and Systems (EuroMLSys '24)*. ACM, 39–46. doi:10.1145/3642970.3655823

[13] Pratyush Patel, Esha Choukse, Chaojie Zhang, Aashaka Shah, Íñigo Goiri, Saeed Maleki, and Ricardo Bianchini. 2024. Splitwise: Efficient Generative LLM Inference Using Phase Splitting. In *Proceedings of the 51st Annual International Symposium on Computer Architecture (ISCA '24)*. IEEE/ACM. doi:10.1109/ISCA59077.2024.00019

[14] Ruoyu Qin, Zheming Li, Weiran He, Jialei Cui, Feng Ren, Mingxing Zhang, Yongwei Wu, Weimin Zheng, and Xinran Xu. 2025. Mooncake: Trading More Storage for Less Computation — A KVCache-centric Architecture for Serving LLM Chatbot. In *Proceedings of the 23rd USENIX Conference on File and Storage Technologies (FAST '25)*. USENIX Association. https://arxiv.org/abs/2407.00079

[15] Ying Sheng, Shiyi Cao, Dacheng Li, Banghua Zhu, Zhuohan Li, Danyang Zhuo, Joseph E. Gonzalez, and Ion Stoica. 2024. Fairness in Serving Large Language Models. In *Proceedings of the 18th USENIX Symposium on Operating Systems Design and Implementation (OSDI '24)*. USENIX Association. https://arxiv.org/abs/2401.00588

[16] Jovan Stojkovic, Chaojie Zhang, Íñigo Goiri, Josep Torrellas, and Esha Choukse. 2025. DynamoLLM: Designing LLM Inference Clusters for Performance and Energy Efficiency. In *Proceedings of the 2025 IEEE International Symposium on High-Performance Computer Architecture (HPCA '25)*. IEEE. https://arxiv.org/abs/2408.00741

[17] Biao Sun, Ziming Huang, Hanyu Zhao, Wencong Xiao, Xinyi Zhang, Yong Li, and Wei Lin. 2024. Llumnix: Dynamic Scheduling for Large Language Model Serving. In *Proceedings of the 18th USENIX Symposium on Operating Systems Design and Implementation (OSDI '24)*. USENIX Association. https://arxiv.org/abs/2406.03243

[18] Jiahao Wang, Jinbo Han, Xingda Wei, Sijie Shen, Dingyan Zhang, Chenguang Fang, Rong Chen, Wenyuan Yu, and Haibo Chen. 2025. KVCache Cache in the Wild: Characterizing and Optimizing KVCache Cache at a Large Cloud Provider. *Proceedings of the 2025 USENIX Annual Technical Conference (ATC '25)* (2025). https://arxiv.org/abs/2506.02634

[19] Yuxin Wang, Yuhan Chen, Zeyu Li, Xueze Kang, Yuchu Fang, Yeju Zhou, Yang Zheng, Zhenheng Tang, Xin He, Rui Guo, Xin Wang, Qiang Wang, Amelie Chi Zhou, and Xiaowen Chu. 2025. BurstGPT: A Real-World Workload Dataset to Optimize LLM Serving Systems. In *Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining (KDD '25), Volume 2*. ACM. doi:10.1145/3711896.3737413

[20] Yuxing Xiang, Xue Li, Kun Qian, Yan Zhang, Wenyuan Yu, Ennan Zhai, Xin Jin, and Jingren Zhou. 2026. ServeGen: Workload Characterization and Generation of Large Language Model Serving in Production. In *Proceedings of the 23rd USENIX Symposium on Networked Systems Design and Implementation (NSDI '26)*. USENIX Association. https://www.usenix.org/conference/nsdi26/presentation/xiang-servegen

[21] Mert Yildiz, Alexey Rolich, and Andrea Baiocchi. 2026. Dispatching policies in data center clusters: Insights from Google and Alibaba workloads. *Performance Evaluation* 172 (2026), 102551. doi:10.1016/j.peva.2026.102551

[22] Gyeong-In Yu, Joo Seong Jeong, Geon-Woo Kim, Soojeong Kim, and Byung-Gon Chun. 2022. Orca: A Distributed Serving System for Transformer-Based Generative Models. In *Proceedings of the 16th USENIX Symposium on Operating Systems Design and Implementation (OSDI '22)*. USENIX Association. https://www.usenix.org/conference/osdi22/presentation/yu

[23] Netanel Zakay and Dror G. Feitelson. 2014. Workload resampling for performance evaluation of parallel job schedulers. *Concurrency and Computation: Practice and Experience* 26, 12 (2014), 2079–2105. doi:10.1002/cpe.3240

[24] Yinmin Zhong, Shengyu Liu, Junda Chen, Jianbo Hu, Yibo Zhu, Xuanzhe Liu, Xin Jin, and Hao Zhang. 2024. DistServe: Disaggregating Prefill and Decoding for Goodput-Optimized Large Language Model Serving. In *Proceedings of the*

*18th USENIX Symposium on Operating Systems Design and Implementation (OSDI '24)*. USENIX Association. https://arxiv.org/abs/2401.09670

[25] Kan Zhu, Mathew Jacob, Chenxi Ma, Yi Pan, Stephanie Wang, Arvind Krishnamurthy, and Baris Kasikci. 2026. TraceLab: Characterizing Coding Agent Workloads for LLM Serving. *arXiv preprint* (2026). https://arxiv.org/abs/2606.30560